

Pull Sequence of Known Length

```
library(Biostrings)
#library(ShortRead)
dna <- readDNAStringSet("~/input folder/sequencing_read_forward_R1.fastq",
                        format="fastq", nrec= -1L, skip=0L, seek.first.rec = TRUE, use.names = TRUE,
                        with.qualities = FALSE)
dna = dna[width(dna)==300]
writeXStringSet(dna, "~/output folder/sequencing_read_forward_R1_300.fasta",
                append = FALSE, compress = FALSE, compression_level = NA, format = "fasta")
```

150 Screening

```
library(Biostrings)
#library(ShortRead)
dna <- readDNAStringSet("/Users/lab/folder/sequencing_read_forward_R1.fastq",
                        format="fastq", nrec= -1L, skip=0L, seek.first.rec = TRUE, use.names = TRUE,
                        with.qualities = FALSE)
# extract indices of those > 150
```

```
I = width(dna) > 325
```

```
# trim only those
```

```
dna[I] = subseq(dna[I],start=1,end=325)
writeXStringSet(dna, "/Users/lab/output folder/sequencing_read_forward_R1trimmed.fastq",
                append = FALSE, compress = FALSE, compression_level = NA, format = "fasta")
```

R1 Translation Script

```
#Two step R1 Nterm translation script
mygencodeTAGB <- GENETIC_CODE
```

```
mygencodeTAGB["TAG"] <- "B"
Lpattern <- "TG"
```

```
writeXStringSet(translate(trimLRPatterns(Lpattern= Lpattern, subject =
readDNAStringSet("/Users/lab/input
folder/sequencing_read_forward_R1trimmed.fastq",format="fastq", nrec= -1L, skip=0L,
seek.first.rec = TRUE, use.names = TRUE), max.Lmismatch = 10.0), if.fuzzy.codon = "solve",
genetic.code =
mygencodeTAGB),"/Users/lab/desktop/folder/sequencing_read_forward_R1trimmed",append=FALSE,
compress=FALSE, compression_level = NA, format = "fasta")
```

```
#Call a DNA sequence from your file
gr <- as(seqinfo(fastqfile), "GRanges")getSeq(fastqfile, gr[3])
```

```
#print a sequence from your file
print(getSeq(fastqfile, gr[3]))
```

Reformat 2 line Fasta into 1 line

```
awk '/^>/ {printf("%s%s\t", (N>0?"\n":""), $0); N++; next;} {printf("%s", $0);} END
{printf("\n");}' < /Users/lab/sequencing_read_forward_R_ trimmed_translated.fasta >
/Users/lab/sequencing_read_forward_R1trimmed_translated.fasta
tr "\t" "\n" < /Users/lab/sequencing_read_forward_R1trimmed_translated.fasta
```

R2 Translation Script

#Two step R2 Cterm translation script that translates and trims the reverse complement of the file that is read in

```
mygenecodeTAGB <- GENETIC_CODE
```

```
mygenecodeTAGB["TAG"] <- "B"
```

```
Lpattern <- "T"writeXStringSet(translate(trimLRPatterns(Lpattern = Lpattern, subject =
reverseComplement(readDNAStringSet("/Users/lab/folder/sequencing_read_forward_R2trimmed.fasta"),
format="fastq", nrec= -1L, skip=0L, seek.first.rec = TRUE, use.names = TRUE)),
max.Lmismatch = 10.0),if.fuzzy.codon = "solve", genetic.code = mygenecodeTAGB),
"/Users/lab/output folder/sequencing_read_forward_trimmed_translatedR2",append = FALSE,
compress = FALSE, compression_level = NA, format = "fasta")
```

```
#Call a DNA sequence from your file
gr <- as(seqinfo(fastqfile), "GRanges")getSeq(fastqfile, gr[3])
```

```
#print a sequence from your file
print(getSeq(fastqfile, gr[3]))
```

Filter by Amino Acid Sequence

#This version is optimized for Windows file directory

```
fasta_list = []
```

```
with open('/Users/lab/input folder/ sequencing_read_forward_R1trimmed_translated.fasta') as
fasta_file:
```

```

for line in fasta_file:
    fasta_list.append(line)

MPIA_list_genes = []
MPIA_list_seqs = []
No_MPIA_list_genes = []
No_MPIA_list_seqs = []

for i in range(len(fasta_list)):
    if ">" not in fasta_list[i]:
        if "BACKBONE AMINO ACIDS" in fasta_list[i]:
            MPIA_list_genes.append(fasta_list[i-1])
            MPIA_list_seqs.append(fasta_list[i])
        if "BACKBONE AMINO ACIDS" not in fasta_list[i]:
            No_MPIA_list_genes.append(fasta_list[i-1])
            No_MPIA_list_seqs.append(fasta_list[i])

with open('/Users/lab/outputfolder/sequencing_read_forward_R1trimmed_translatedGOOD.fasta','w')
as f:
    for i in range(len(MPIA_list_genes)):
        f.write(MPIA_list_genes[i])
        f.write(MPIA_list_seqs[i])

with open('/Users/lab/outputfolder/sequencing_read_forward_R1trimmed_translatedBAD.fasta','w')
as f:
    for i in range(len(No_MPIA_list_genes)):
        f.write(No_MPIA_list_genes[i])
        f.write(No_MPIA_list_seqs[i])

```

Parse CD-hit script

```

import sys
import collections
import re
import csv

out1 = sys.argv[1] # aa1234_S1_out.txt
out_clstr = sys.argv[2] # aa1234_S1_out.txt.clstr

```

```

removeStopCodon = sys.argv[3] # yes

print(out1)
print(out_clstr)
#parser = argparse.ArgumentParser(description="")
#parser.add_argument("UKB", help="Path ")
#parser.add_argument("--output-file", help="File ")
#parser.add_argument("--filter", help=" ")
#args = parser.parse_args()
#with open(args.filter, "r") as stream:
# fields = list(set([line.strip().split(",")[0] for line in stream]))
#subfields = list(set([line.strip() for line in stream]))

# store out1
dict1 = {}
with open(out1, "r") as stream:
    temp = None
    for line in stream:
        line = line.strip()
        if re.search('^>.+$', line) != None:
            temp = line
        else:
            dict1[re.sub("^(.+)\s.+$", "\g<1>", temp)] = [temp, line]

# store out_clstr (only header, frequency and * sequence)
dict2 = {}
with open(out_clstr, "r") as stream:
    header = None
    repre = None
    freq = None
    aa = None
    for line in stream:
        line = line.strip()
        if re.search('^>.+$', line) != None:
            if freq != None:
                dict2[header] = [int(freq), aa, repre]
            header = line # new header
        else:
            freq, li = line.split("\t")
            if re.search(".*\*$", li) != None:
                aa, repre = li.split(" ")
                repre = re.sub(r"(.+)\.\.\.\*$", "\g<1>", repre)

```

```

# at the end of file
dict2[header] = [int(freq), aa, repre]

# order by frequency
dict2 = collections.OrderedDict(sorted(dict2.items(), key=lambda kv: kv[1], reverse=True))

# export sequence
#with open('parsed_'+out1+'.csv', 'w',newline='') as outfile:
with open('parsed_'+out1+'.csv', 'w') as outfile:
    wr = csv.writer(outfile,delimiter='\t')
    header = ['ClusterID','Frequency','Sequence','Length','LibraryID']
    _ = wr.writerow(header)
    for key,value in dict2.items():
        s = dict1[value[2]]
        l = [key, value[0], s[1], value[1], s[0]]
        if removeStopCodon=='yes':
            if re.search("\*",s[1])==None:
                _ = wr.writerow(l)
        else:
            _ = wr.writerow(l)

```